MICHAEL LEHN, Ph.D.

Scientific Software Engineer · C/C++ · Compilers & LLVM · High-Performance Computing

Ulm, Germany · Open to relocation to the United States or Canada

michael.lehn@uni-ulm.de · LinkedIn · GitHub · Ulm University

SUMMARY

Software engineer and mathematician (Ph.D.) with 20+ years of hands-on experience developing performance-critical C/C++ software. Expertise spans numerical algorithms and high-performance linear algebra, memory hierarchy and cache-aware algorithms, SIMD, multithreaded and distributed computing, CPU architecture, compiler construction, and low-level systems programming. Recent projects include cross-architecture BLAS micro-kernels (x86-64, ARM SME2, CUDA), an LLVM-based compiler, an early-stage linear algebra compiler on MLIR, and event-driven FPGA control for quantum experiments.

TECHNICAL SKILLS

Languages: C++ (C++11–20), C, Python, Assembly (x86-64, AArch64, custom RISC), LLVM IR, SystemVerilog

Performance & HPC: Numerical linear algebra, BLAS, LAPACK, ScaLAPACK, SIMD micro-kernels (SSE3, NEON, SME2), cache-aware/blocked algorithms, std::thread, OpenMP, MPI, CUDA/Tensor Cores, benchmarking and profiling

Compilers & Systems: LLVM, compiler construction, code generation, x86-64 System V ABI, C interoperability, ISA design, CPU architecture, FPGA (iCE40, Artix-7/Sinara), bare-metal programming

Tools: Linux/Unix, macOS, Git, CMake, GDB, Valgrind, perf, Intel VTune, NVIDIA Nsight, Verilator, cocotb, Yosys

SELECTED PROJECTS (OPEN SOURCE)

ulmBLAS **— High-Performance BLAS Library** C++ · Assembly · CUDA

- Designed a complete BLAS library (levels 1–3, four data types) on the layered BLIS architecture: portable C++ blocking and packing with replaceable micro-kernels; tested with the Netlib BLAS and Reference-LAPACK suites.
- Wrote SSE3, NEON, and SME2 kernels and integrated BLIS, AOCL, and OpenBLAS kernels; with OpenBLAS's kernel, single-threaded DGEMM reaches ~97% of OpenBLAS and ~99% of oneMKL (Ice Lake, n ≥ 1000).
- Experimental Apple M5 SME2 backend: DGEMM up to 490 GFLOP/s, 1.06–1.15× Apple Accelerate for n ≥ 1050; Accelerate is still faster for medium sizes.
- Release 0.4.0 (November 2026): shared libraries that select SME2/NEON or AVX-512/AVX2/SSE3 kernels at load time, so NumPy and SciPy run on ulmBLAS unchanged. For Julia: ulmBLAS_jll built with Yggdrasil and registered, and UlmBLAS.jl (registration in progress), which makes ulmBLAS Julia's BLAS via libblastrampoline.
- In progress: LU and Cholesky factorizations, solves, and inverses as stride-generic C++ templates, parallelized with PLASMA-style OpenMP tasks; GEMM with bfloat16 and FP16 inputs.
- CUDA DGEMM tutorial (in progress): Tensor Core kernels reach 0.82 of cuBLAS on an NVIDIA A100 (n ≥ 1000). Authored the GEMM Tutorial (referenced by FLAME); ulmBLAS is used in Meta Quest software.

ABC **("A Better C") — Systems Language & LLVM Compiler** C++ · LLVM

- Designed a small systems language exposing C's machine model with simplified declaration syntax; built an LLVM-based compiler with interchangeable backends targeting LLVM IR and custom ULM assembly.
- Implemented native C interoperability with x86-64 System V ABI lowering, enabling direct use of existing C libraries without wrappers; demonstrated with raylib applications.
- Isolated a wrong-code bug in LLVM's optimizer and reported it upstream with a minimal reproducer (#235074).
- Wrote not-abc, a small compiler first implemented in ABC, then rewritten in its own language (self-hosting).

FLENS **— Flexible Library for Efficient Numerical Solutions** C++

- Built a numerical computing framework combining MATLAB-like abstractions with direct access to optimized BLAS/LAPACK kernels, achieving zero abstraction overhead as verified by benchmarks and generated assembly.
- Generic architecture: algorithms run unchanged on local, distributed, GPU-resident, and implicit matrices; compile-time C++ techniques map operations to optimized kernels and detect costly temporaries.

ulm.lam **— Linear Algebra Compiler on MLIR (early prototype)** C++ · MLIR · Python · Julia

- Developing a compiler that translates linear algebra formulas written in Julia or Python into BLAS/LAPACK calls, using declared matrix properties to choose factorizations, exploit symmetry, and avoid explicit inverses.
- Built two MLIR dialects, for matrix expressions and for BLAS/LAPACK kernels, with passes for property inference, kernel selection, and workspace reuse; generates C++ for ulmBLAS, Julia's BLAS, or any CBLAS/LAPACK. Automates the formula-to-kernel mapping I implemented by hand in FLENS-LAPACK.
- In initial generalized least-squares benchmarks (n = 2000, m = 50), runtime is comparable to an expert-written Julia version; temporaries live in a workspace reused across calls, so each call allocates KiB instead of MiB.

ulm-on-iCE / ulm-generator **— FPGA CPU & ISA Toolchain** SystemVerilog · C/C++ · Assembly

- Designed and implemented a custom RISC CPU in SystemVerilog on an iCE40 FPGA, from digital logic and ISA design to assembly programs running on physical hardware.
- Defined a declarative ISA description format and built a generator producing an assembler, virtual machine, instruction-level debugger, and documentation from a single specification.

qubic-notes **— QubiC Compiler, ISA & RTL Simulation** Python · Verilog · cocotb

- Studied LBNL's open-source QubiC qubit-control system; documented its compiler and ISA and built QD, a prototype quantum-control DSL with explicit timing and synchronization, compiled through QubiC-IR to machine code and verified in cycle-accurate RTL simulation.

PROFESSIONAL EXPERIENCE

Ulm University — Institute of Numerical Mathematics · Ulm, Germany 2008 – present

Senior Lecturer (Akademischer Oberrat, permanent appointment), Numerical Mathematics

- Develop and maintain the open-source projects above and own their full engineering lifecycle: architecture, implementation, testing, performance benchmarking and profiling, documentation, and long-term maintenance.
- Design and teach courses in HPC, compiler construction, systems programming, and numerical mathematics; the HPC curriculum goes from memory hierarchy and cache-aware algorithms through SIMD micro-kernels and multithreading to MPI and GPUs.
- Built a full-stack computing curriculum spanning logic gates, registers and ALUs, ISA design, assembly, systems programming, and compiler construction, supported by custom-developed software and FPGA hardware.

Ulm Center for Scientific Computing (UZWR), Ulm University · Ulm, Germany 2006 – 2008

Managing Director

- Developed complete C++/Qt applications for industrial and research partners, translating their engineering and scientific problems into mathematical models and working numerical software.
- Among many projects, built a self-contained Windows application for MRI-based biomechanical analysis, the complete stack in-house, from FEM algorithms to the user interface.
- Scoped and evaluated incoming projects, designed computational approaches, and either implemented solutions directly or coordinated them with researchers possessing the required expertise.

Ulm University — Institute of Numerical Mathematics · Ulm, Germany 2002 – 2008

Research Associate

- Developed FLENS, a C++ library for high-performance numerical computing, as the core of my doctoral research.

University of Southern California — Modeling and Simulation Laboratory · Los Angeles, CA 1999 – 2001

Research Assistant / Teaching Assistant

- Applied mathematics and scientific computing with Prof. Chunming Wang in USC's Modeling and Simulation Laboratory; master's research in collaboration with NASA's Jet Propulsion Laboratory (JPL).

SELECTED INDUSTRY & RESEARCH COLLABORATIONS

German Aerospace Center (DLR) — Real-Time Control / Quantum Technologies 2025 – present

- Collaborate with the DLR Institute of Quantum Technologies on real-time control of a space-ready atomic-gas quantum memory; analyzed the ARTIQ/Sinara architecture and proposed FPGA logic for latency-critical reactions.
- Develop HeraldRT, an event-driven SystemVerilog controller for Sinara Kasli (Artix-7): a herald photon switches lasers off via a processor-free hardware path; a synchronous path time-stamps and sequences the experiment.
- Simulated (800 self-checks), synthesized, placed, and routed with the open-source openXC7 flow, and generated a bitstream; hardware tests, latency and jitter measurements, and Vivado timing sign-off are pending.

Tensor AI Solutions — HPC / Tensor Networks 2024

- Collaborated with Tensor AI Solutions on Tartarus, a C++ HPC library for tensor contractions in tree-tensor-network applications for machine learning and QML; co-supervised the student project and contributed substantially to its numerical algorithms and performance-oriented design.
- Applied TTGT and Loops-over-GEMM contraction schemes, BLAS kernels, and C++ code generation in Python.

Daimler TSS (Daimler Group) — Machine Learning / Recurrent Neural Networks 2017 – 2018

- Collaborated with Daimler TSS on the mathematical analysis of training recurrent neural networks (vanishing and exploding gradients); co-supervised a research-driven bachelor's thesis.

EDUCATION

Ph.D., Numerical Mathematics — Ulm University, Germany 2008

Dissertation: FLENS — A Flexible Library for Efficient Numerical Solutions · Advisor: Prof. Karsten Urban

M.S., Mathematics — Ulm University, Germany 2002

M.S., Applied Mathematics — University of Southern California, Los Angeles, CA 2001